

An Instruction-Set Extension to Support Approximate Multicore Processors

Daniela Catelan¹, Felipe Sovernigo¹, Liana Duenha¹, Ricardo Santos¹

Abstract—Approximate computing encompasses several techniques that can be applied from the application level to the circuit level, offering better performance and lower power consumption at the cost of a lack of precision. Multicore architectures, by distributing the workload among several processing cores, can meet the demand for performance while remaining within the energy limitations. This work focuses on the usage of heterogeneous multicore designs, comprised of approximate cores. Specifically, this paper proposes an extension to RISC-V Instruction-Set Architecture (ISA) that supports approximate operations. We have designed a sub-set of approximate integer and floating point instructions that, in turn, are embedded in the operations of mathematical functions (sine, cosine, tangent, exponential, and logarithmic). The proposed technique is named Instruction-Level Approximate Functions (ILAF) and it has been evaluated on accuracy, number of instructions, cycles, and power in six applications using the ACCEPT compiler framework and the SPIKE RISC-V simulator. We have performed experiments comparing our approach to non-approximate and approximate versions of each application. The results show significant improvements, such as a 94.18% reduction in power consumption in the CUBIC application and 78.39% cycles reduction in the IDENTITY_LOG2 application. Additionally, a detailed and comprehensive analysis of ILAF and a software-based approximate technique (FastApprox) in the BLACKSCHOLE application showed that ILAF achieved a reduction in power consumption of $3.02\times$ with 19% code coverage.

Index Terms—approximate computing, multicore, heterogeneous processors, approximate instructions, ILAF

I. INTRODUCTION

Processor designers face challenges in delivering high performance while staying within limited power and cost budgets. The end of Dennard scaling and the difficulty of adding processing hardware and maintaining the same power envelope made new alternatives necessary. One alternative is Approximate Computing (AC), which provides techniques ranging from the application to the circuit level [1].

AC is helpful for tasks where a margin of error in the result can be tolerated in exchange for a benefit in power consumption and performance increase. Most AC techniques aim to solve specific problems or require programmer intervention to identify which parts of an application are suitable for approximations [2].

Designers have also been asked to design multicore architectures that meet the demand for computational performance while remaining within power constraints by distributing the workload across multiple processing cores. Each core can be optimized for different aspects of performance, allowing the

system to adapt to the needs of applications, offering significant advantages in performance, power, area, and delay [4]. The use of heterogeneous processors combining approximate and exact computing resources is an alternative to maximize performance and reduce power consumption on applications that support some flexibility on the output's accuracy.

There are proposals to meet applications performance and power goals on multicore architecture using approximate techniques [5], [6]. Some approaches control the input voltage or clock frequencies, thus forcing the system to run approximated. Some rely on online software systems to identify approximation hot spots in the application. Our approach identifies approximation opportunities at compile-time and replaces original instructions with approximated ones.

At the software-level approximated techniques, tools such as the ACCEPT [3] compiler are responsible for identifying code regions most suitable for approximation. ACCEPT has a set of approximation techniques at the software level, allowing the programmer to evaluate the impact of the approximations [2] on the application runtime and the accuracy of the results.

The FastApprox (FA) [9] library is a tool that has approximated and vectorized versions of mathematical functions that can be used by applications supporting approximations. Previous experiments with approximate functions in a set of applications achieved accelerations from $1.22\times$ to $634\times$, with a maximum accuracy loss of 30% [2]. These approximate results come only from the software technique since the hardware running the applications is unaware of any approximate approach.

From the previous results on AC applied to mathematical functions, we have observed an opportunity to improve performance and save power consumption, introducing an additional level of approximation by replacing accurate (non-approximate) operations by approximate floating point instructions. This new technique offers a hardware-level (instructions) approximate technique over a source code that is (or is not) already approximated by a software-level technique.

This article introduces the Instruction-Level Approximate Functions (ILAF), which incorporates floating point (FP) approximate instructions into FastApprox's mathematical functions such as sine, cosine, tangent, exponential, and logarithmic. We have extended the ACCEPT compiler, the SPIKE [10] RISC-V simulator, and the RISC-V [11] toolset to support new approximate instructions, generate approximate source code, and simulate approximate applications. The technique has been evaluated on the following metrics: accuracy (relative error), number of instructions, cycles, and power (μW).

¹The authors are with the College of Computing (FACOM) at the Federal University of Mato Grosso do Sul (UFMS), Campo Grande/MS, Brazil. E-mail: {daniela.catelan, felipe.sovernigo, liana.duenha, ricardo.santos}@ufms.br .

The results brought from ILAF and comparing it to approximate mathematical libraries allowed us to identify some contributions of this work:

- A new approximate technique that uses approximate integer and floating point instructions in mathematical functions;
- An extension of the ACCEPT compiler framework, allowing hardware/software approximations using this new technique.
- An infrastructure for approximate experimentation using hardware/software tools: the SPIKE simulator with new approximate instructions, the ACCEPT compiler to use the approximate RISC-V instructions, and the Prof5 profiling tool for power estimation of those instructions.

This article is organized as follows. Section II presents and discusses the related research to this work; Section III presents the Instruction-Level Approximate Function Design from the floating point approximate instruction design up to the approximate mathematical functions design and implementation; Section IV describes the applications and the experiments carried out to evaluate and validate the approximation technique; The results and discussion are presented in Section V; Finally, the conclusions are in Section VI.

II. RELATED WORK

The papers [5], [7], and [8] analyze different approaches to improve the energy efficiency and performance of multicore systems through approximate computing techniques. Together, these papers show the potential of approximate computing to optimize the trade-off between energy consumption and performance, each approaching the problem with different techniques.

In [5], a technique called Approx-RM aims at approximating iterative applications on heterogeneous multicore platforms. Approx-RM predicts, at runtime, the number of iterations required to achieve the desired precision, slightly relaxing the precision target to reduce the number of iterations while saving energy. In addition, it dynamically adjusts resource allocation, such as voltage frequency (DVFS), core type, and number of cores, to meet execution deadlines while minimizing energy consumption. Approx-RM was evaluated in the ARM big.LITTLE platform, achieving average energy savings of 31.6%, with a precision reduction of only 1%. Furthermore, time and energy overheads were less than 0.1%.

A new approach to improve multicore processors' performance through configurable approximate arithmetic units (AAUs) is proposed in [7]. The work proposes a processor architecture incorporating approximate floating-arithmetic units with different precision settings. These units can be dynamically adjusted according to the precision and performance needs of the applications while operating under power (TDP) constraints. The Sniper simulator for multicore processors with eight cores and benchmarks from the AxBench library was used. Tests were performed using three configurations of approximate floating-arithmetic units, varying the precision to analyze the impact on performance and energy consumption.

The approach showed that the multicore system can execute error-resilient applications up to 19% faster than precise systems without exceeding the TDP limits.

The work [8] presents the AISC approach: Approximate Instruction Set Computer, proposing a new heterogeneous computing platform called AISC, where each computing unit (core) supports a subset of the same instruction set. Although the ISA subsets are not functionally complete individually, their union provides a complete and functional ISA for the platform as a whole, allowing the simplification of the microarchitecture of each computing unit, reducing its complexity, and improving energy efficiency. An implementation is presented exploring ISA simplification techniques in terms of Depth (exclusion of complex instructions) and Breadth (reduction of instruction complexity, such as reduced precision). The authors used benchmarks to evaluate the impact of AISC techniques in terms of instruction count, instruction mix, energy consumption, and precision loss. The AISC platform demonstrated a reduction of up to 37% in energy consumption with a precision loss of approximately 10%.

When comparing the proposal of this work with those presented previously, one may observe that, unlike other approaches, the proposal seeks to add an instruction level of approximate computing by identifying and replacing accurate (non-approximate) mathematical operations with approximate floating point instructions. The ILAF approach does not rely on dynamic overheads to identify code hot-spots to approximate or changing clock frequencies or power supply. ILAF seems to be a promising alternative as a first approximation step on accurate (non-approximate) functions or even as a second approximation step on software-level approximated mathematical functions.

In the era of high-performance computing, the quest for greater efficiency, reduced power consumption, and accuracy in arithmetic operations is a constant challenge, especially in floating point and fixed point computing, where balancing accuracy, performance, and power consumption is crucial. The innovative works [30], [31], and [32] delve into these areas, focusing on pioneering techniques for keeping the accuracy while optimizing the runtime and power consumption of floating point, mixed-precision, and fixed point operations.

In [30], noise propagation models in floating point (FP) AC are discussed, proposing metrics and design frameworks to evaluate and improve the accuracy and performance of approximate calculations. The study introduces a design framework that helps predict and mitigate the effects of errors, enabling more robust development of AC systems. This method includes Pre-Approximation (PAM), which induces zero-mean noise with uniform distribution and facilitates the construction of noise propagation models for floating point operations such as addition, subtraction, and multiplication. Based on these models, metrics were defined to estimate the quality of applications, determining the efficient bit-width required for mantissa and suitability for truncation.

The FlexFloat tool is presented in [31] aiming to improve the flexibility and efficiency of floating point operations,

dynamically adjusting the precision and format of the data. By controlling the bandwidth of the mantissa and exponent fields, the results showed that FlexFloat can significantly reduce energy consumption and runtime while maintaining or improving performance compared to traditional models. On an embedded GPU, mixed-precision computing enabled by FlexFloat enabled an average reduction from 20% up to 52% in execution time and from 22% up to 60% in power consumption.

In [32], the focus is on developing a library of fixed point trigonometric functions for high-level synthesis using the CORDIC algorithm [21]. This approach offers accuracy comparable to floating point solutions but with lower resource consumption and better suitability for FPGA implementations. The paper demonstrates that the fixed point library has almost the same accuracy, with errors ranging between 0.1% and 0.4%, while significantly reducing the hardware size.

The previous work share a common focus on optimizing arithmetic operations to control accuracy and improve performance and energy efficiency in computing systems. All explore techniques that balance the accuracy with efficient performance, through modeling and mitigating noise in floating point computing, the dynamic flexibility of floating point, or the efficient implementation of fixed point trigonometric functions using the algorithm CORDIC.

There are yet other approaches using AC techniques but acting straightly on the mathematical functions accuracy and performance instead of looking only at the operations. The authors in [20], [23], and [24] propose techniques and tools for approximating mathematical functions, all aimed at optimizing computational efficiency while maintaining precision.

The work in [20] analyzes different methods for approximating mathematical functions such as sine, cosine, tangent, exponentials, and logarithms. It explores techniques including shift-and-add algorithms (such as CORDIC), polynomial or rational approximations, table-based methods, and bit manipulation (Mitchell's algorithm [22]). When choosing an approximate computing technique, the paper considers the trade-off between computational efficiency and accuracy. It compares the techniques, evaluating versatility, speed, precision, scalability, and suitability for hardware or software implementations. The authors main conclusion is that although the methods aim to obtain quick estimates with controlled loss of precision, the choice of the ideal method will depend on the mathematical function to be approximated, the desired precision, and possible table size restrictions. As an example, a square root approximation using an adjusted constant reduced the maximum relative error from 0.0607 to 0.03476.

The authors in [23] presented a fine-tuning structure to perform approximations on trigonometric functions. A library of sine and cosine functions was developed. They introduced a new code generation approach, FixM, to minimize code duplication, supporting dynamic fixed point algorithm design without compromising accuracy, and relied on an automated precision tuning framework and a code generator, using the CORDIC algorithm, to approximate mathematical functions.

The compiler replaces the so-called function origins with their specialized versions. Tests were carried out using benchmarks InverseK2J, FFT, and FBench running on two different microcontrollers (ARM Cortex versions) to demonstrate the approach effectiveness in the sin and cos functions. The result showed a significant performance improvement, with an acceleration of up to 180% and energy savings of up to 60% at an insignificant error cost.

The Puppeteer framework is proposed in [24] and uses uncertainty quantification techniques, such as Global Sensitivity Analysis (GSA) [25], to evaluate applications sensitivity to approximation errors. Puppeteer allows developers to identify insensitive regions within various benchmarks and apply approximate computation, resulting in significant performance improvements in applications such as HPCCG, DCT, and Blacksholes. The authors applied Global Sensitivity Analysis (GSA) and Uncertainty Quantification (UQ) [28] methods to measure the sensitivity of application outputs to errors in code blocks. The framework also employs advanced UQ libraries such as SALib [26] and VARS-TOOL [27] to generate error domain samples and evaluate code regions sensitivity. The authors also apply the FastApprox library [9] to provide approximate calculation functions such as SQRT, LOG, EXP, and CNDF, significantly accelerating data processing with acceptable relative errors. By implementing approximate versions of benchmarks, the framework has achieved notable performance improvements. It successfully identified insensitive regions, and applying approximate computation to these areas led to performance increases of $1.18\times$ for HPCCG, $1.18\times$ for DCT, and $1.3\times$ for Blacksholes.

III. INSTRUCTION-LEVEL APPROXIMATE FUNCTIONS DESIGN

As part of the ILAF design workflow, we adopted a three-level space exploration (DSE) approach (Figure 1). At the first level, we focused on how to explore the design of new floating point (FP) approximate instructions (step 1). The focus is analyze all the set of operations that comprise a complex floating point and replace some of these operations by their approximate versions to improve performance and/or power consumption while keeping controlled accuracy levels. After having new approximate floating point instructions, the second level (step 2) works in the source code of mathematical functions, identifying non-approximate FP instructions that could be replaced with approximate ones. Again, the building of the new approximate mathematical function is guided by the gains in performance and reduced power consumption and looking at the accuracy losses. At the third level (step 3), we aimed to identify and evaluate the functions in the applications' source code and determine which ones could be replaced with the approximate mathematical functions. Steps 1 and 2 are presented in the following subsections. The mapping of the approximate functions into some applications' source code (step 3) is presented in Section IV.

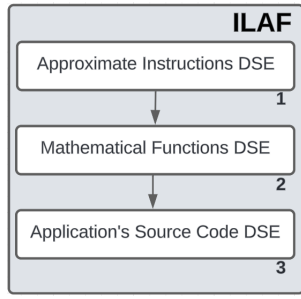


Fig. 1: ILAF design workflow.

A. Floating Point Approximate Instructions Design

ILAF has adopted the RISC-V instruction-set as the reference ISA to design and evaluate new approximate instructions. The FP instructions in RISC-V use integer operations to manipulate exponents, mantissa, shift, and rounding. Our strategy to design new approximate FP instructions was to exchange some original (accurate) integer operations for approximate integer instructions. The decision-making on which integer operations and instructions should be replaced was done after performing a DSE on those operations and evaluating the impact in accuracy, power, and performance.

The approximate integer operations adopted in this work were first introduced in our work [1], [12]. The approximate integer addition follows the RISC-V instruction-set format and is implemented using the InXA1 [13] approximate adder. The approximate integer subtraction was designed and implemented using the APSC4 approximate subtractor [14].

Figure 2 presents the approximated hardware block diagram proposed in this work for FP addition and subtraction instructions. The highlighted block (BIG ALU) was replaced with 6 approximate integer add operations, thus becoming a BIG ALU APPROX block. The approximate add FP instruction that uses this new hardware is named **FADDX**. The design of the new **FSUBX** instruction has also changed the BIG ALU hardware block and it uses three approximate integer operations (two add and one sub)¹.

The new six approximate integer addition operations used in the BIG ALU of the approximate FP addition hardware block are shown in Figure 3. Lines 1, 4, 7, and 10 present the original (non-approximate) integer addition operations (represented by the sign “+”), and lines 2, 5, 8, and 11 present the approximate operations addition sign (“^”). A similar procedure was performed with the **FSUBX** instruction, as can be seen in Figure 4, where lines 1, 5, and 8 present integer operations, and lines 2, 6, and 9 feature the exchange for approximate instructions. It should be noted that the original integer subtraction operation (“-”) is replaced by the logical function of the APSC4 subtractor.

¹The implementations of the approximate integer and floating point instructions, as well as the steps for their installation, are available at: <https://github.com/danielacatlan/Approximate-Instructions>.

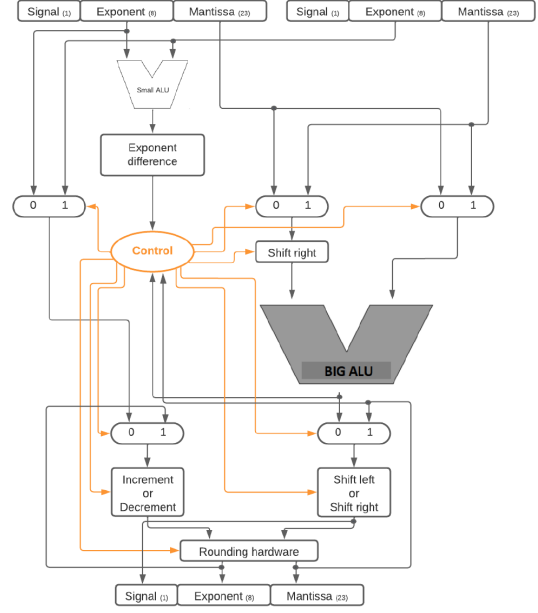


Fig. 2: Block diagram of the approximate FP addition and subtraction hardware.

```

1 //sigZ = 0x01000000 + sigA + sigB;//ORIG
2 sigZ = 0x01000000 ^ sigA ^ sigB;//APPROX
3
4 //sigA = sigA + expA ? 0x20000000 : sigA;//ORIG
5 sigA = sigA ^ expA ? 0x20000000 : sigA;//APPROX
6
7 //sigB = sigB + expB ? 0x20000000 : sigB;//ORIG
8 sigB = sigB ^ expB ? 0x20000000 : sigB;//APPROX
9
10 //sigZ = 0x20000000 + sigA + sigB;//ORIG
11 sigZ = 0x20000000 ^ sigA ^ sigB;//APPROX

```

Fig. 3: Approximate addition operations used in the BIG ALU of the approximate FP addition hardware.

```

1 //sigDiff = sigA - sigB;//ORIG
2 sigDiff = ((~sigA & sigB) | (sigB & Cin) |
3 (~sigA & Cin) | (sigA & ~sigB & ~Cin));//APPROX
4
5 //sigY = sigA + (expA ? 0x40000000 : sigA);//ORIG
6 sigY = sigA ^ (expA ? 0x40000000 : sigA);//APPROX
7
8 //sigY = sigB + (expB ? 0x40000000 : sigB);//ORIG
9 sigY = sigB ^ (expB ? 0x40000000 : sigB);//APPROX

```

Fig. 4: Approximate subtraction operations used in the BIG ALU of the approximate FP subtraction hardware.

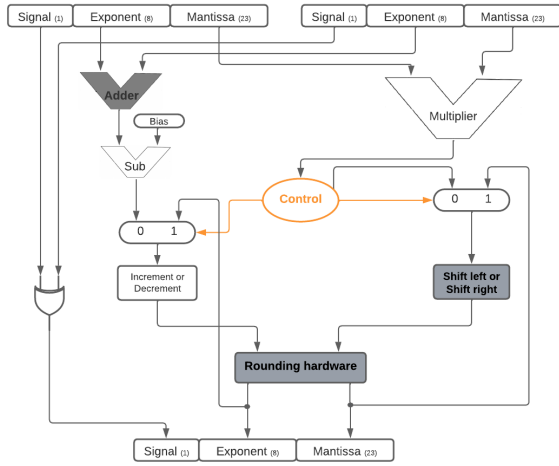


Fig. 5: Block diagram of the approximate FP multiplier hardware.

```

1 //expZ = expA + expB - 0x7F; //ORIG
2 expZ = expA ^ expB - 0x7F; //APPROX
3
4 aux = ((uint_fast64_t) 1<<dist); //ASSIST
5 //return a>>dist | (a & (aux - 1)) != 0); //ORIG
6 approx_sub = ((~aux & 1) | (1 & 0) | (~aux & 0)
7 | (aux & ~1 & ~0)); //APPROX
8 return a>>dist | ((a & approx_sub) != 0); //APPROX
9
10
11 //isTiny = (softfloat_detectTininess ==
12 softfloat_tininess_beforeRounding) || (exp < -1)
13 || (sig + roundIncrement < 0x80000000); //ORIG
14 isTiny = (softfloat_detectTininess ==
15 softfloat_tininess_beforeRounding) || (exp < -1)
16 || (sig ^ roundIncrement < 0x80000000); //APPROX
17
18 //sig = (sig + roundIncrement) >> 7; //ORIG
19 sig = (sig ^ roundIncrement) >> 7; //APPROX

```

Fig. 6: Approximate multiply operations of the approximate FP multiplier hardware.

Figure 5 presents the new extended FP hardware block diagram for multiplication. The new approximate FP multiplication **FMULX** instruction has three approximate add operations (Figure 6, lines 2, 13, and 17) and one approximate subtraction operations (figure 6, line 6). The Adder block (add the 2 exponents) has one approximate add operation, becoming an approximate exponent adder block. The other approximate integer operations are on the mantissa side (Shift left or Shift right block) and rounding hardware block.

The divider FP instruction block diagram (Figure 7) is similar to the multiplication diagram, subtracting the exponents first and then adding to the Bias. The exponent subtraction block (Sub), the mantissa shift block (Shift left or Shift right), and the rounding block have two approximate addition operations and one approximate subtraction operation, as shown in Figure 8 (lines 2, 5, and 9). This set of new operations comprises the new **FDIVX** approximate instruction.

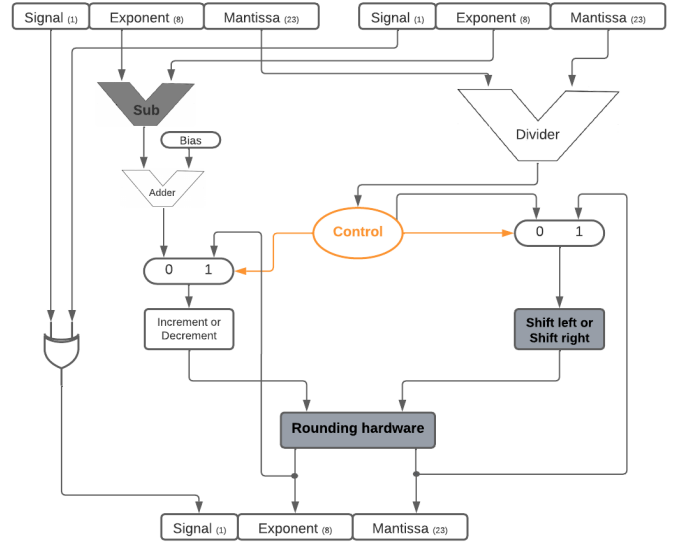


Fig. 7: Block diagram of the approximate FP divider hardware.

```

1 //expZ = expA - expB + 0x7F; //ORIG
2 expZ = expA - expB ^ 0x7F; //APPROX
3
4 //sigZ = sigZ + 2; //ORIG
5 sigZ = sigZ ^ 2; //APPROX
6
7
8 //sigZ = sigZ - 4; //ORIG
9 sigZ = ((~sigZ & 4) | (4 & 0) | (~sigZ & 0)
10 | (sigZ & ~4 & ~0)); //APPROX

```

Fig. 8: Approximate division operations of the approximate FP divider hardware.

B. Approximate Functions Design

The second step of the design and implementation of ILAF is built on the top of the ACCEPT compiler using the following mathematical functions of the FastApprox library (FA): SIN, COS, TAN, EXP, LOG, and LOG₂. When the user starts the compilation process, ACCEPT presents a list of mathematical functions able to apply approximate optimization. The user may choose 3 options: non-approximate, *fast* (small error), and *faster* (considerable error). The first option uses the non-approximate mathematical functions from the *math.h* library. *fast* and *faster* options apply versions the approximated functions from the FastApprox library [2], [9].

We redesign the mathematical functions of FastApprox (*fast* version) by replacing some of the original FP instructions to the new approximate instructions (presented in subsection III-A). As an example, Figure 9(a) presents a code snippet of the FastLog₂ function (log₂ approximate mathematical function of FA). In the FastLog₂ function, replacing the original FP division operation/instruction (line 6) to the new **FDIVX** approximate FP instruction (Figure 9(b)) could meet the compromise between accuracy, power, and performance.

The preliminary experiments improved the number of cycles, instructions, and power, keeping the same accuracy of the original FastLog₂ function.

```

1 static inline float fastlog2 (float x)
2 {
3     ...
4     return
5     y-124.22551499f-(1.498030302f*mx.f)-
6     (1.72587999f / (0.3520887068f+mx.f));
7 }

```

(a) FastLog₂ function.

```

1 static inline float fastlog2 (float x)
2 {
3     ...
4     return
5     y-124.22551499f-(1.498030302f*mx.f)-
6     FDIVX(1.72587999f,(0.3520887068f+mx.f));
7 }

```

(b) FastLog₂ with **FDIVX** instruction.

Fig. 9: Code snippet of the FastLog₂ function.

Each mathematical function implemented in the *fast* version of FA was submitted to the same design procedure: original FP instructions were replaced by approximate FP instructions. The decision-making on the number of FP instructions to be replaced and where (in the functions' source code) to be placed has considered the results of a comprehensive set of DSE experiments. Each experiment is a scenario, a combination of candidate FP instructions, where the approximate instructions were placed in the functions. Each possible scenario has been evaluated looking at the accuracy, performance increase, and power consumption reduction.

From all the evaluated functions, the TAN function had the most significant number of scenarios (10). The chosen scenario was based on the best (lesser) mean absolute percentage error (MAPE) value:

$$MAPE = \frac{1}{n} \sum_{i=1}^n \left| \frac{E_i - A_i}{E_i} \right| \quad (1)$$

where:

- n is the sample size;
- E_i is the original (accurate) value/output;
- A_i is the approximate value/output.

Table I displays the scenario (the amount of approximate instructions) and its MAPE value for each mathematical function. The best scenario of the SIN function had the most significant number of approximate FP instructions (three **FADDX** and three **FMULX**). The COS and TAN functions had one **FADDX** instruction, the LOG and LOG₂ functions had one **FDIVX** instruction and the EXP function had one **FSUBX**.

TABLE I: Selected approximate instructions and MAPE of each mathematical function.

Functions	FADDX	FSUBX	FMULX	FDIVX	MAPE
SIN	3		3		0.15
COS	1				0.03
TAN	1				0.14
EXP		1			0.00
LOG				1	0.00
LOG ₂				1	0.00

IV. EXPERIMENTAL SETUP

ILAF has been evaluated and tested on six different applications that apply mathematical functions²: CUBIC [17], FF-BENCH [15], FBENCH [15], BLACKSCHOLE [16], LOG2 and IDENTITY_LOG2 (author himself).

The type of mathematical functions present in each of the applications, the number of functions, the number of functions replaced by ILAF (number in parenthesis), and the number of executions (bold) are summarized in Table II. For example, the CUBIC application presents three cosine functions, each one run once, but only two were replaced by ILAF.

TABLE II: Summary of mathematical functions in each application.

APPLICATIONS	SIN	COS	TAN	EXP	LOG	LOG ₂
CUBIC	-	3 (2) 3	-	-	-	-
FFBENCH	2 (2) 64	-	-	-	-	-
FBENCH	6 (0) 12	2 (2) 2	1 (0) 4	-	-	-
BLACKSCHOLE	-	-	-	3 (3) 6	4 (4) 4	-
LOG2	-	-	-	-	-	1 (1) 50
IDENTITY_LOG2	-	-	-	-	-	2 (2) 5000

Figure 10 illustrates the ACCEPT compiler approximate workflow where a user can choose between the FA or the ILAF approaches. Given a source code (step 1), the ACCEPT compiler will analyze the code and identify all mathematical functions able to approximation (step 2). In step 3, the user will choose the approximate function model. When choosing ILAF (step 4), the mathematical functions of the source code will be replaced by the ones from the FA with approximate instructions. In step 5, the mathematical functions will be from the FastApprox library. Step 6 translates the assembly to machine code RISC-V. Step 7 is the simulation on an approximate core using the SPIKEX simulator. SPIKEX represents an instance of a RISC-V core, built on top of the SPIKE simulator and it supports all the approximate instructions we have designed in this work.

The performance and power results were acquired from the Prof5 [18] tool. Prof5 is a RISC-V profiling that uses the SiFive E24 RV32IMAFBC microcontroller. Prof5 allows the

²Applications are available for download at: <https://github.com/lscad-facom-ufms/ILAF-ApproxFunction>.

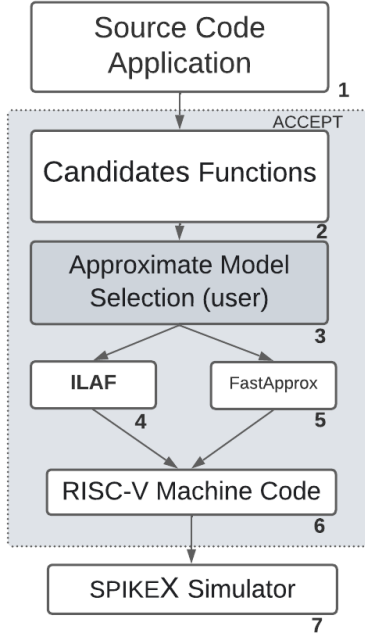


Fig. 10: ACCEPT approximate functions workflow.

user to create detailed profiles of RISC-V programs from the SPIKEX log, generating profiles that include the number of cycles, instructions, and power consumption of each instruction and function on a core. The Prof5 energy model was customized to calculate the power of all approximate instructions. The approximate integer instructions have an average power gain (compared to the non-approximate instruction) of 1.3% [1], single-precision approximate FP instructions have an average power gain of 1.2% [19].

Table III presents the results from the Prof5 tool on the number of cycles, power (μW) and power difference (%) of non-approximated and approximated instructions. Columns 2-3 present the number of cycles and power of the non-approximate instructions. The number of cycles is the same for non-approximation and approximate instructions. Column 5 shows the power of the approximate instructions designed in this work. Equation 2 was used to calculate the power difference in percentage ($\text{Power}_{\text{Diff}}$) between the non-approximated instructions ($\text{Power}_{\text{nap}}$) and approximated (Power_{ap}) and the results are presented in column 6. Since approximate instructions have lower power values than exact instructions, the more approximate instructions in an application, the greater the power improvement.

$$\text{Power}_{\text{Diff}} = \frac{(\text{Power}_{\text{nap}} - \text{Power}_{\text{ap}})}{\text{Power}_{\text{nap}}} \times 100 \quad (2)$$

V. RESULTS AND DISCUSSION

The results and discussion presented in this section are based on a set of applications organized into three different

TABLE III: Cycles and power results of non-approximate and approximate instructions.

Instructions	Cycles	Power _{nap} (μW)	Instructions	Power _{ap} (μW)	Power Difference (%)
add	1	2.80	addx	2.76	1.43
sub	1	2.86	subx	2.82	1.40
mul	1	3.09	mulx	3.05	1.29
div	1	3.09	divx	3.05	1.29
fadd	2	3.36	faddx	3.18	5.36
fsub	2	3.43	fsubx	3.34	2.62
fmul	2	3.71	fmulx	3.52	5.12
fdiv	2	3.71	fdivx	3.61	2.70

versions: the baseline (BL) where the applications have non-approximate mathematical functions; the FastApprox (FA), where the applications had the approximate functions from the FastApprox library and the ILAF approach where the applications used ILAF approximate functions built on the top of the FastApprox library. Applications from BL and FA are run on an accurate (non-approximate) RISC-V core. Applications compiled using the ILAF approach are run on an approximate core. The experiments evaluated accuracy, running cycles, and power (μW) comparing the original (non-approximate) application to the FA and ILAF.

The relative error ($RE = \frac{|AO - BO|}{|BO|}$) is the metric adopted to evaluate the application accuracy when using the approximate functions optimization. RE is calculated from the baseline output results (BO) representing the non-approximate applications. The Approximate Output (AO) is the output result of the ILAF or the FA approaches. In the experiments, only two applications (CUBIC and FBENCH) had a maximum $RE = 0.3$ (30%). The remaining applications presented $RE = 0$, meaning no loss of accuracy using the approximate functions approaches.

The results shown that the approximate techniques directly impact applications highly dependent on the math functions. Other applications may have limited benefits when looking at the performance and power consumption of the whole running code. Table IV shows the percentage improvement of instructions, power, and cycles in each application by using ILAF over BL.

TABLE IV: Percentage improvements of the applications running the ILAF approach compared to the BL.

APPLICATIONS	Instructions	Power	Cycles
CUBIC	94.77	94.18	95.19
FFBENCH	0.02	0.01	0.01
FBENCH	63.10	56.83	76.01
BLACKSCHOLE	88.17	88.71	90.81
LOG2	49.08	23.38	66.62
IDENTITY_LOG2	76.95	77.16	78.39

An improvement of 94.77% in the number of instructions can be seen in the CUBIC application compared to the baseline (BL) version, meaning that 94.77% fewer instructions in the cosine function using our ILAF approach. Figure 11

sketches the functions called by the cosine function in each version of the application. One may notice that there is a large difference between instructions (in parenthesis) of the cosine function in Figure 11(a) and Figure 11(c). This discrepancy was caused by the additional functions brought to the application by the *math.h* library. In Figure 11(a), CUBIC runs three calls to cosine that, in turn, calls three other functions (`_ieee754_rem_pio2`, `Kernel_sin`, and `Kernel_cos`). Figure 11(b) represents the CUBIC using the approximate cosine function (`fast_cos`) from the FastApprox library. When using ILAF (Figure 11(c)), CUBIC performs one calls to `fast_cos` and two calls to `fast_cos1`, which is the cosine from ILAF.

An analysis was performed regarding the applications code region where ILAF was applied. The goal was to observe how many instructions (at runtime) of the math functions of FastApprox could be impacted by ILAF. The code coverage metric is presented in equation 3, where T_{ILAF} is the number of ILAF approximate instructions applied in a mathematical function. (T_I) is the total of instructions of a given FastApprox function. The power improvement ($Power_{imp}$) is also calculated in equation 4. Each instruction of the set of replaced instructions (I_R) of a given function is multiplied by its power ($Power_i$), and I_{Approx} is the set of FP approximate instructions of ILAF that replaces the instructions of I_R . The speedup (equation 5), is represented by the runtime (in cycles) of the FastApprox function by the runtime of the ILAF function.

$$Coverage = \frac{T_{ILAF}}{T_I} \quad (3)$$

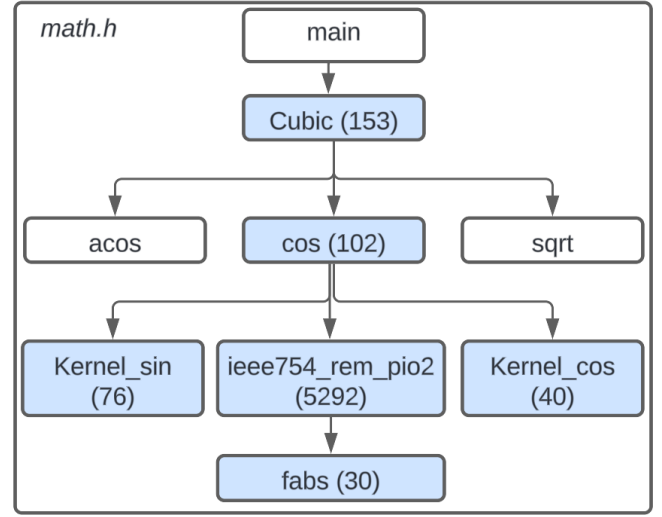
$$Power_{imp} = \frac{\sum_i^{I_R} i \times Power_i}{\sum_j^{I_{Approx}} j \times Power_j} \quad (4)$$

$$Speedup_{Cycle} = \frac{\sum_i^{I_R} i \times Cycle_i}{\sum_j^{I_{Approx}} j \times Cycle_j} \quad (5)$$

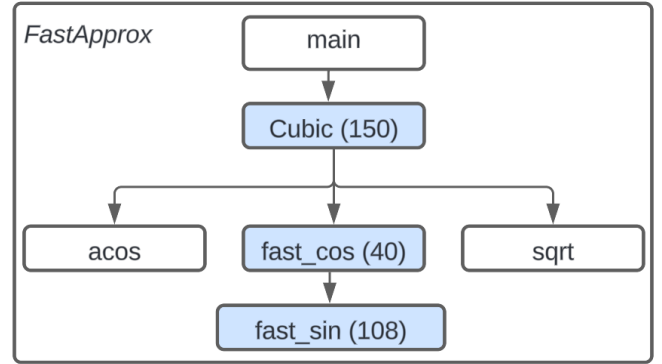
Table V displays the coverage, power improvement, and speedup cycle comparing ILAF to FastApprox. The results are based on the instructions of the mathematical functions that are impacted by our approach.

The coverage achieved in the FFBENCH application means that the ILAF approximate instructions covers 30.60% of all instructions (at runtime) of the FastApprox sine function. Specifically, FFBENCH uses two sine functions which are run $64 \times$. Each sine function of ILAF has three **FADDX** and three **FMULX** instructions, so that 384 ($6 \times 64 = 384$) instructions, at runtime, were replaced.

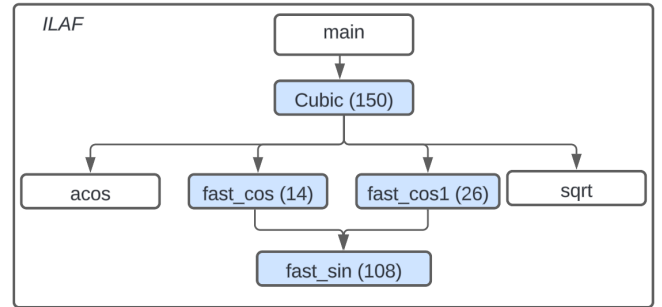
When running the CUBIC application with FA, the `fast_cos` function performs three add operations to calculate the cosine.



(a) Cosine function call flow in the *math.h* library.



(b) Cosine function call flow in the FastApprox library.



(c) Cosine function call flow in the ILAF.

Fig. 11: Call flow of the cosine function of CUBIC using *math.h*, FastApprox, and ILAF.

TABLE V: Coverage, power improvement, and speedup of ILAF compared to FA.

APPLICATIONS	Coverage (%)	Power Improvement	Speedup Cycles
CUBIC	7.50	1.58	1.50
FFBENCH	30.60	1.03	1.00
FBENCH	1.30	1.04	1.00
BLACKSCHOLE	19.00	3.02	1.00
LOG2	4.00	1.03	1.00
IDENTITY_LOG2	3.80	1.03	1.00

The design of the ILAF cosine function has only two adds, thus achieving a cycle speedup of $1.5\times$.

The BLACKSCHOLE application had the largest number of functions impacted by the ILAF technique (three exponentials and four logarithms). The larger use of the ILAF approximate functions is the responsible for the best power improvement ($3.02\times$) among the applications. From the applications set in this work, BLACKSCHOLE is also the application with most opportunities to explore the performance on multiple cores since it is part of the PARSEC benchmark. We do not carry out experiments evaluating parallel applications running on multiple cores since the SPIKE simulator does not support running applications on instances of heterogeneous multiple cores.

Table VI shows a code snippet of the `transit_surface` function that is part of the cosine function of FastApprox. Table VII depicts the `transit_surface` function of the ILAF approach. Both Tables highlight some floating point instructions, the number of running instructions, power and cycles. The FBENCH application source code has 6 sine functions, 2 cosines, and 1 tangent, but the best results are found when ILAF is applied only on the cosine functions. Both functions have the same number of instructions (72) in the code snippet, but ILAF reduces the total power consumption (calculated by multiplying the number of instructions by power). FastApprox had 12 `fadd.s` instructions, while ILAF replaced 8 with `faddx.s`, resulting in 4 remaining `fadd.s` instructions.

TABLE VI: Excerpt from the code of the “`transit_surface`” function (FBENCH application) using the *FastApprox* library.

<code>transit_surface:</code>	Instructions	Power	Cycles
<code>:</code>	<code>:</code>	<code>:</code>	<code>:</code>
<code>fadd.s</code>	12	3.36	2
<code>fdiv.s</code>	4	3.71	2
<code>flt.s</code>	8		
<code>c.bnez</code>	20		
<code>ret</code>	16	1.82	2
<code>c.j</code>	12		

VI. CONCLUSIONS

Approximate computing enables performance improvements and energy savings at the cost of a lack of precision.

TABLE VII: Excerpt from the code of the “`transit_surface`” function (FBENCH application) using *ILAF*.

<code>transit_surface:</code>	Instructions	Power	Cycles
<code>:</code>	<code>:</code>	<code>:</code>	<code>:</code>
<code>fadd.s</code>	4	3.36	2
<code>fdiv.s</code>	4	3.71	2
<code>flt.s</code>	8		
<code>c.bnez</code>	20		
<code>faddx.s</code>	8	3.19	2
<code>ret</code>	16	1.82	2
<code>c.j</code>	12		

The design of heterogeneous processors maximize efficiency and performance by distributing the workload across different cores. This paper proposes an extension to the RISC-V ISA that integrates approximate computing with multicore design, providing new approximate instructions to a processor core.

The approach Instruction-Level Approximate Function (ILAF), incorporates approximate instructions into mathematical functions such as sine, cosine, tangent, exponential, and logarithmic. The technique was built on top of the ACCEPT compiler framework and the FastApprox library.

Experiments were carried out on applications with mathematical functions, looking at the impacts of the ILAF approach and comparing them to non-approximate applications (baseline) and an approximate approach using the FastApprox library. Application CUBIC using ILAF achieved a power improvement of 94.18% and 95.19% reduction in the cycles compared to the baseline. The BLACKSCHOLE application using ILAF achieved a power improvement of $3.02\times$ with only 19% code coverage. These results showed that ILAF significantly improved performance and power consumption compared to the non-approximate and even software-level approximate versions of the applications. The comparison to the FastApprox library revealed that our approach of approximate instructions provides power improvement for all the applications, keeping the levels of accuracy achieved by the software technique (FastApprox).

The adoption of ILAF poses a challenging task to attain better performance, power improvement, and keeping accepted levels of accuracy. The benefits of ILAF relies on the right decision-making on approximate instruction replacement in the mathematical functions. This challenge comes from the difficulty of performing the design space exploration, given that this task has an exponential complexity. This work applied design space exploration in three levels: the design of the approximate floating point instructions, the design of the approximate functions, and the design of the approximate applications. We had to explore the space of the floating point instructions to choose the proper replacements of the original floating point instructions to the approximate ones. This exploration has a straight impact on the core design. Also, it was necessary to perform the design space exploration

in the mathematical functions to identify the instructions that the approximate ones should replace. Lastly, a third exploration step was performed in the applications to evaluate which mathematical functions should be replaced with their approximate counterparts.

Comparing to the related work, ILAF is the only approach that embeds approximate instructions into mathematical functions, thus adding a hardware-level approximation to software approximate mathematical functions. Although other AC techniques are applied to mathematical functions, ILAF stands out for its practicality and versatility, in addition to presenting performance results, and can be used either as a first step to approximate functions or as a second step to refine mathematical functions already approximated in software.

ILAF is an ongoing work and it presents many opportunities for further research investigation. The technique can be extended to a larger set of mathematical functions (*math.h* has more than 100 floating point functions) thus encompassing more applications that support approximations and could take advantage of a hardware/software approach. Another possibility for future research is evaluating the technique on simulators supporting the design of heterogeneous cores to evaluate applications that could take advantage of parallelism among approximate functions.

ACKNOWLEDGMENTS

The authors thank Brazilian Research Agencies FUNDECT, CAPES, and CNPq, and UFMS for their financial support to the Research Laboratory of High Performance Computing Systems (LSCAD). This study was financed in part by the Coordenação de Aperfeiçoamento de Pessoal de Nível Superior - Brasil (CAPES) - Finance Code 001.

REFERENCES

- [1] D. Catelan, L. Duenha, R. Santos. *Evaluation and characterization of approximate arithmetic circuits*. CCPE, 2022. pp. e6865.
- [2] L. Reis, L. Wanner. *Functional Approximation and Approximate Parallelization with the ACCEPT compiler*. Proc. of the IEEE 33rd SBAC-PAD, pp. 188-197, 2021.
- [3] A. Sampson, A. Baixo, B. Ransford, T. Moreau, J. Yip, L. Ceze, M. Oskin. *Accept: A programmer-guided compiler framework for practical approximate computing*. University of Washington Technical Report UW-CSE-15-01, vol. 1, pp. 1-14, 2015.
- [4] Tulika Mitra, *Heterogeneous Multi-core Architectures*, IPSJ Transactions on System and LSI Design Methodology, vol. 8, pp. 51-62, 2015, doi: 10.2197/ipsjtsldm.8.51
- [5] M. W. Azhar, M. Manivannan, P. Stenström. *Approx-RM: Reducing Energy on Heterogeneous Multicore Processors under Accuracy and Timing Constraints*. ACM Transactions on Architecture and Code Optimization. 20(3). ACM. 2023.
- [6] S. A. K. Gharavi and S. Safari. *Performance Improvement of Processor Through Configurable Approximate Arithmetic Units in Multicore Systems*. IEEE Access, vol. 12, pp. 43907-43917, 2024
- [7] Gharavi, Seyed & Safari, Saeed. (2024). *Performance Improvement of Processor Through Configurable Approximate Arithmetic Units in Multicore Systems*. IEEE Access. PP. 1-1. doi: 10.1109/ACCESS.2024.3380912.
- [8] Alexandra Ferrerón, Jesús Alastruey-Benedé, Darío Suárez-Gracia, Ulya R. Karpuzcu. *AISC: Approximate Instruction Set Computer*. Workshop on Approximate Computing in conjunction with ASPLOS, March, 2018.
- [9] P. Mineiro. *FASTAPPROX library*. Available at: <https://code.google.com/archive/p/fastapprox/>
- [10] Spike RISC-V ISA Simulator. Available at: <https://github.com/riscv-software-src/riscv-isa-sim>
- [11] RISC-V Toolchain. Available at: <https://github.com/riscv-collab/riscv-gnu-toolchain>
- [12] D. Catelan, L. Duenha, L. Wanner, R. Santos. *Instruction-Level Loop Perforation*. Proc. of the WSCAD, 2023, pp. 37-48.
- [13] H. A. F. Almurib, T. N. Kumar, F. Lombardi. *Inexact Designs for Approximate Low Power Addition by Cell Replacement*. Proc of the DATE, pp. 660-665, 2016.
- [14] A. Gorantla, P. Deepa. *Design of Approximate Subtractors and Dividers for Error Tolerant Image Processing Applications*. Journal of Electronic Testing, pp. 1-7, 2019.
- [15] J. Walker. *Floating Point Benchmarks*. Available at: <https://www.fourmilab.ch/fbench/>
- [16] T. DiPasquale (codeslinger). *Black-Scholes Option Pricing Model in C*. Available at: <https://gist.github.com/codeslinger/472083/>
- [17] MiBENCH. *Solve a cubic polynomial*. Available at: <https://github.com/embecosm/mibench>
- [18] J. Silveira, L. Castro, V. Araújo, R. Zeli, D. Lazari, M. Guedes, R. Azevedo, L. Wanner. *Prof5: A RISC-V profiler tool*. Proc of the IEEE 34th SBAC-PAD, pp. 201-210, 2022.
- [19] M. Horowitz. *Computing's energy problem (and what we can do about it)*. Proc. of the IEEE ISSCC, SF, CA, USA, 2014, pp. 10-14.
- [20] Jean-Michel Muller. *Elementary Functions and Approximate Computing*. Proceedings of the IEEE, 2020, v. 108, number. 12, pp. 2136-2149. doi=10.1109/JPROC.2020.2991885.
- [21] J. E. Volder, *The CORDIC Trigonometric Computing Technique*, in IRE Transactions on Electronic Computers, vol. EC-8, no. 3, pp. 330-334, Sept. 1959, doi: 10.1109/TEC.1959.5222693.
- [22] J. N. Mitchell, *Computer multiplication and division using binary logarithms*, IRE Transactions on Electronic Computers, vol. EC-11, no. 4, pp. 512-517, Aug 1962.
- [23] Daniele Cattaneo, Michele Chiari, Gabriele Magnani, Nicola Fosatti, Stefano Cherubin, Giovanni Agosta. *FixM: Code generation of fixed point mathematical functions*. Sustainable Computing: Informatics and Systems, Volume 29, Part B, 2021, 100478, ISSN 2210-5379, <https://doi.org/10.1016/j.suscom.2020.100478>.
- [24] K. Parasyris et al., *Approximate Computing Through the Lens of Uncertainty Quantification*. SC22: International Conference for High Performance Computing, Networking, Storage and Analysis, Dallas, TX, USA, 2022, pp. 1-14, doi: 10.1109/SC41404.2022.00072.
- [25] I. Sobol, *On sensitivity estimation for nonlinear mathematical models*, Matem. Mod., vol. 2, pp. 112-118, 1990. [Online]. Available: <http://mi.mathnet.ru/mm2320>
- [26] J. Herman, W. Usher, *SALib: An open-source python library for sensitivity analysis*, The Journal of Open Source Software, vol. 2, no. 9, jan 2017.
- [27] S. Razavi, R. Sheikholeslami, H. V. Gupta, and A. Haghnegahdar, *Vars-tool: A toolbox for comprehensive, efficient, and robust sensitivity and uncertainty analysis*, Environmental Modelling & Software, vol. 112, pp. 95-107, 2019.
- [28] M. Ye and M. Hill, *Chapter 10 - global sensitivity analysis for uncertain parameters, models, and scenarios*, in Sensitivity Analysis in Earth Observation Modelling, G. P. Petropoulos and P. K. Srivastava, Eds. Elsevier, 2017, pp. 177-210.
- [29] H. Menon, M. O. Lam, D. Osei-Kuffuor, M. Schordan, S. Lloyd, K. Mohror, J. Hittinger, *Adapt: Algorithmic differentiation applied to floating-point precision tuning*, in SC18: International Conference for High Performance Computing, Networking, Storage and Analysis. IEEE, 2018, pp. 614-626.
- [30] Y. Xiang, L. Li, S. Yuan, W. Zhou and B. Guo, *Metrics, Noise Propagation Models, and Design Framework for floating Point Approximate Computing*, in IEEE Access, vol. 9, pp. 71039-71052, 2021, doi: 10.1109/ACCESS.2021.3053578.
- [31] G. Tagliavini, A. Marongiu and L. Benini, *FlexFloat: A Software Library for Transprecision Computing*, in IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, vol. 39, no. 1, pp. 145-156, Jan. 2020, doi: 10.1109/TCAD.2018.2883902.
- [32] Naohiro Iwanaga, Takayoshi Abe, and Akira Yamawaki, *Development of Fixed-point Trigonometric Function Library for High-level Synthesis*. 2013. the 1st IEEE/IIAE International Conference on Intelligent Systems and Image Processing. pp. 91-94. doi: 10.12792/icisip2013.021.